



系统编程

Systems Programming

第一章 系统编程导论



授课时间 第一周

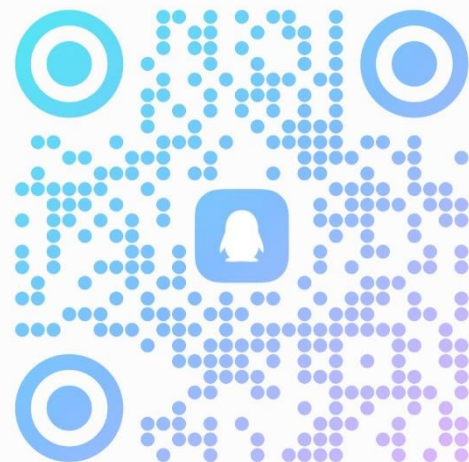
冯禹洪
电话: 15016740789

- 1.1 什么是系统调用
- 1.2 什么是系统编程
- 1.3 开发环境与工具
- 1.4 初始化OSChatBox



2026系统编程

群号: 932293497



扫一扫二维码, 加入群聊





本章学习目标：

奠基系统编程认知与实践基础



项目拆解

构建OSChatBox开发与运行环境

项目里程碑

系统初始化
目录结构与初始源码框架

知识目标

- ❖ 理解系统调用的核心概念与作用
- ❖ 掌握系统编程的基本特点与重要性
- ❖ 熟悉openEuler开发环境与工具链
- ❖ 了解即时通信系统的基本模型

能力目标

- ❖ 搭建完整的C语言开发与调试环境
- ❖ 掌握基本的Shell命令
- ❖ 掌握GCC、GDB、Make等工具
- ❖ 能够初始化现场即时通信软件



系统编程三大支柱

系统调用

操作系统原理

开发实践

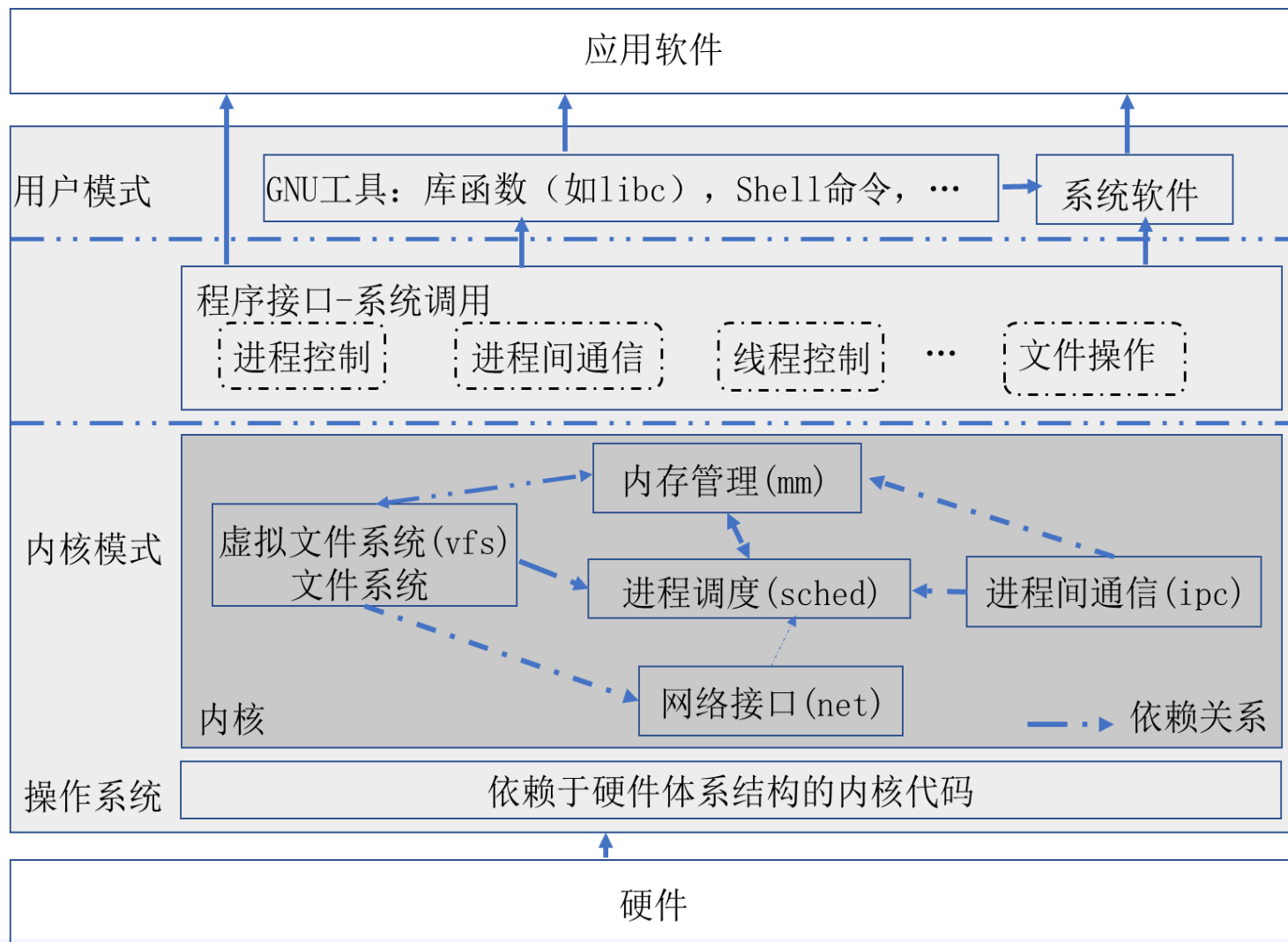
课程知识体系全景图





1.1

什么是系统调用



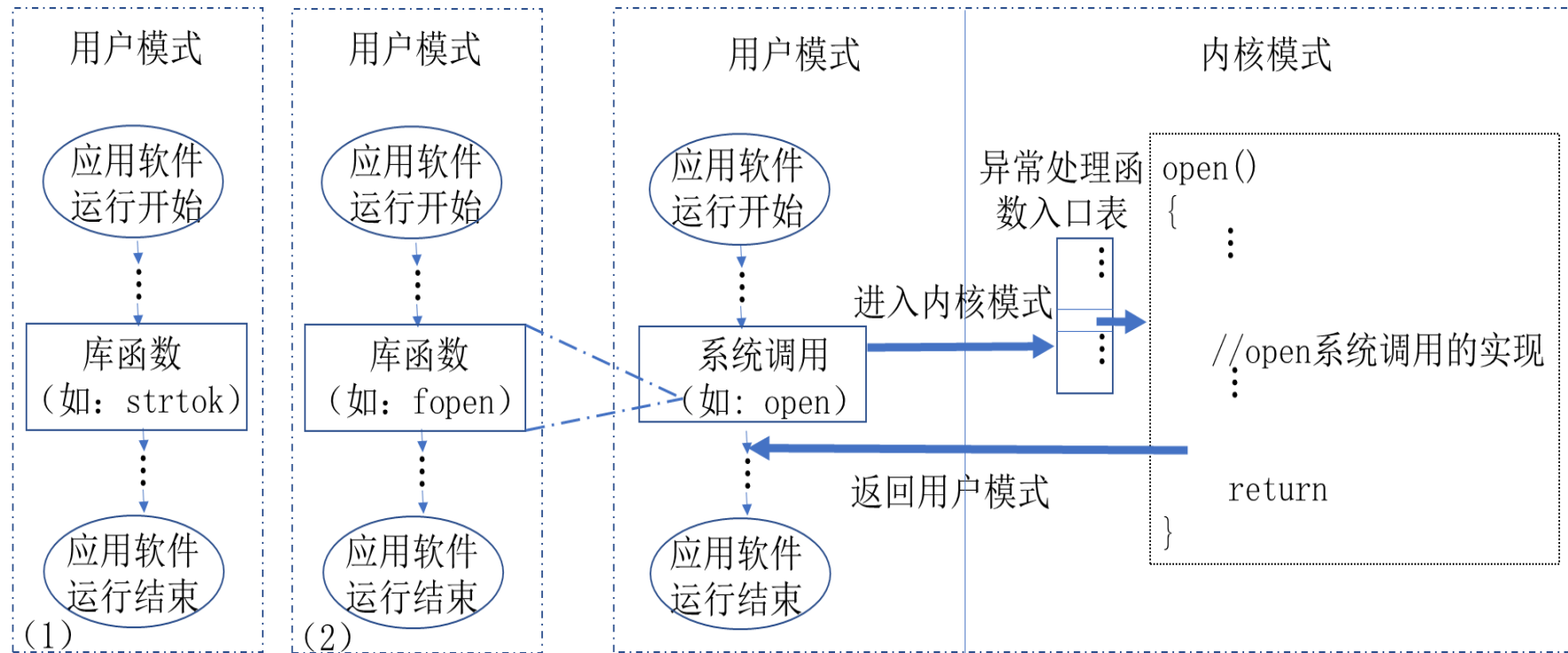
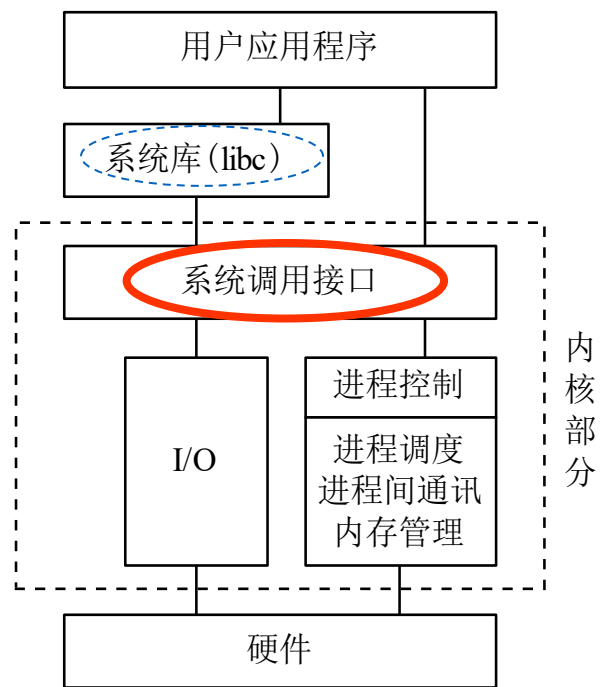
我们的程序在这里运行

唯一的桥梁

掌控一切资源



系统调用的特点



(a) 调用库函数

(b) 调用系统调用



系统调用的特点

特权操作

访问硬件、受保护资源

稳定接口

不同硬件，相同接口

性能开销

用户态/内核态切换

错误处理

通过返回值报告状态，通过`errno`获得出错原因

系统调用示例

类别	代表调用	功能说明
文件操作	open/read/write/close	文件创建、读写、关闭

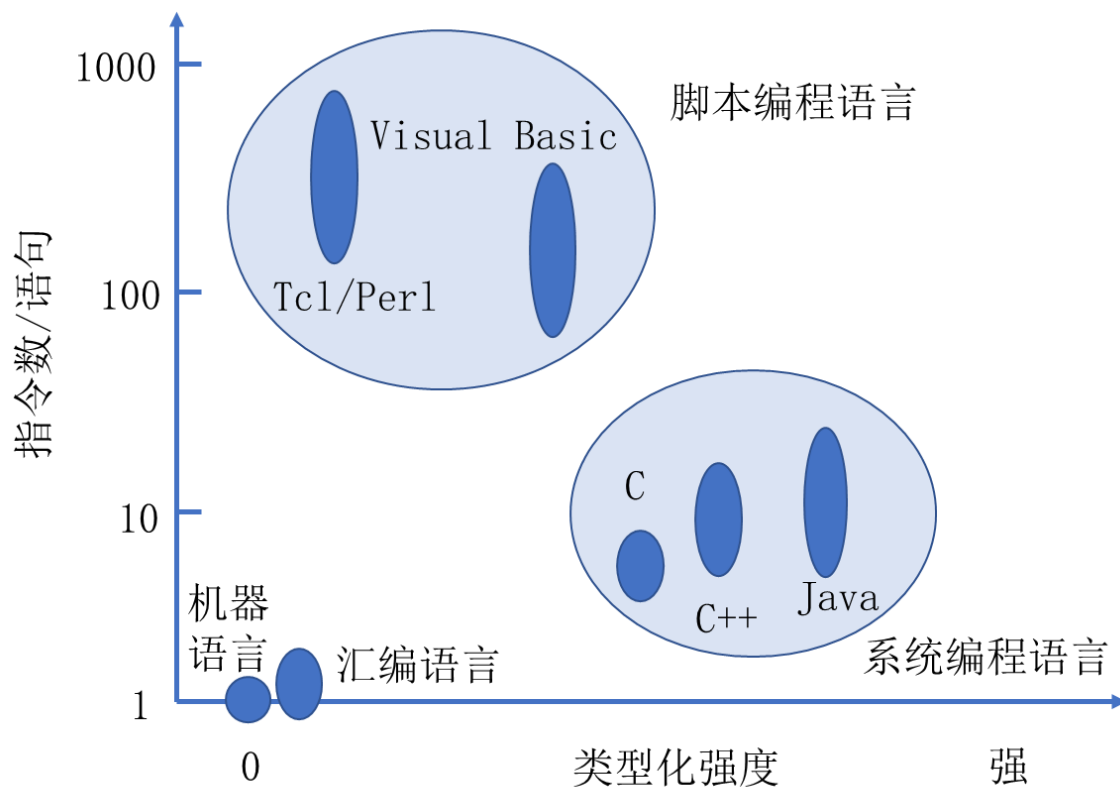


1.2

什么是系统编程？



基于级别和类型化强度的编程语言比较（1998）



应用软件

VS

系统软件

维度	应用软件	系统软件
典型例子	办公软件、游戏、Web 应用	操作系统、驱动、数据库
运行环境	运行时环境之上	接近操作系统内核
关注点	业务逻辑 用户体验	工程复杂性高 资源高效性 高持续可用性 高可延展性 高可靠性 严密安全性
主要语言	设备/平台独立：Java、Python、JavaScript	汇编、C/C++、Rust、Java (Hadoop)、Scala (Spark)
错误处理	依赖异常处理机制	必须检查每个系统调用



为什么要学习系统编程？

1. 产业需求

【快Star-X】通用计算服务器系统设计工程师

全职 | 工程类 - 系统架构 | 北京 | 2026-01-16

职位描述

- 1、负责分析业务场景平台架构，输出系统设计指标，驱动机型设计；
- 2、负责主导服务器系统（计算/存储/高速互联）的架构设计、开发及调优；
- 3、负责建立硬件性能与稳定性测试体系，覆盖兼容性、稳定性等；
- 4、负责大规模服务器硬件监控系统的开发与优化；
- 5、负责开发性能分析方法与平台，软硬件一体化系统架构设计；
- 6、负责主导关键部件（CPU/SSD/加速器）的定制化研究与落地；
- 7、负责推进X86/AMD/ARM平台的应用迁移与性能优化，构建跨架构能力矩阵

任职要求

- 1、本科及以上学历，计算机系统、体系结构、软件工程、通信、网络及相关专业；
- 2、熟练掌握Linux环境下的C/C++/Python/Shell/PHP/Java/Golang等1种以上语言，并具有熟练的编程功底，**熟练掌握Linux系统基本操作，了解系统内核、文件系统、进程；**

Tencent 腾讯招聘

社会招聘校园招聘生活在腾讯产品和服务

搜索工作岗位

操作系统高级研发工程师（深圳/北京/上海） 分享 收藏

深圳

TEG | 技术 | 五年以上工作经验 | 更新于2026年01月20日

岗位职责

负责操作系统基础软件源码维护，需求开发，上游反馈等；

2.负责操作系统核心组件设计和研发，腾讯自研软件开发和支持；

3.负责软件包分析，打包和编译，软件包依赖解决；**岗位要求**

1.硕士及以上学历，精通一种或者多种软件开发语言（C/C++、Python、Go等），五年及以上系统软件或者平台软件开发经验；

2.熟悉操作系统架构设计和核心组件，具备主导系统软件设计和核心组件开发；

HUAWEI

本岗位有多个领域，请你选择一个具体的方向。

AI软件开发

岗位职责

1、负责AI领域的软件工程化和产品开发；

2.负责AI算法及系统的设计和实现，包括但不限于：神经网络与机器学习、计算机视觉、无人驾驶、路径规划、智能决策、推荐系统、大模型、生成式AI等；

3.负责产品的集成和调测，以及各类工具链的开发；**岗位要求**

1、计算机或人工智能相关专业，具备一个及以上大型实际软件项目经验，独立承担过关键模块的开发工作；

2.熟练运用至少一门编程语言（C++ /Java /Python/Rust等）；

熟悉Linux系统、了解系统内核、具备实际软件项目



为什么要学习系统编程？



2. 计算机系统能力人才的需求

构建底层核心能力	理解操作系统核心机制：进程/线程、通信与同步、资源调度
	掌握并发程序设计：解决多任务环境下的协调与效率问题
	具备系统级分析与设计能力：从系统视角理解程序如何运行
培养系统化工程思维	实践系统级开发全流程：设计 → 实现 → 部署 → 维护
	培养系统架构思维：构建模块化、可扩展、可靠的软件系统 - 理解边界：用户态 vs. 内核态 - 避免漏洞：缓冲区溢出、竞争条件
形成高效的系统实现与调优能力	编写高性能系统程序：开发高效、稳定、可维护的底层软件 操作系统 驱动 运行时库 应用程序 系统编程在这里
	精通系统调试与优化：掌握多进程/线程环境的问题诊断与性能调优



1.3

开发环境与工具 - 实验环境

操作系统发展进程



国产操作系统发展进程



本课程实验环境

服务器

TaiShan 2280 V2
鲲鹏920处理器
ARM64架构
48内核

虚拟机

NUMA节点: 1
socket: 2
2核/socket
共4核

操作系统

openEuler
内核版本: 4.19.90

华为闭环生态: 服务器处理器 + 终端设备 + OS, 硬件产品的更新 - > OS的升级

课程案例均基于TaiShan服务器 (鲲鹏处理器) + openEuler系统环境进行实现与测试



1.3

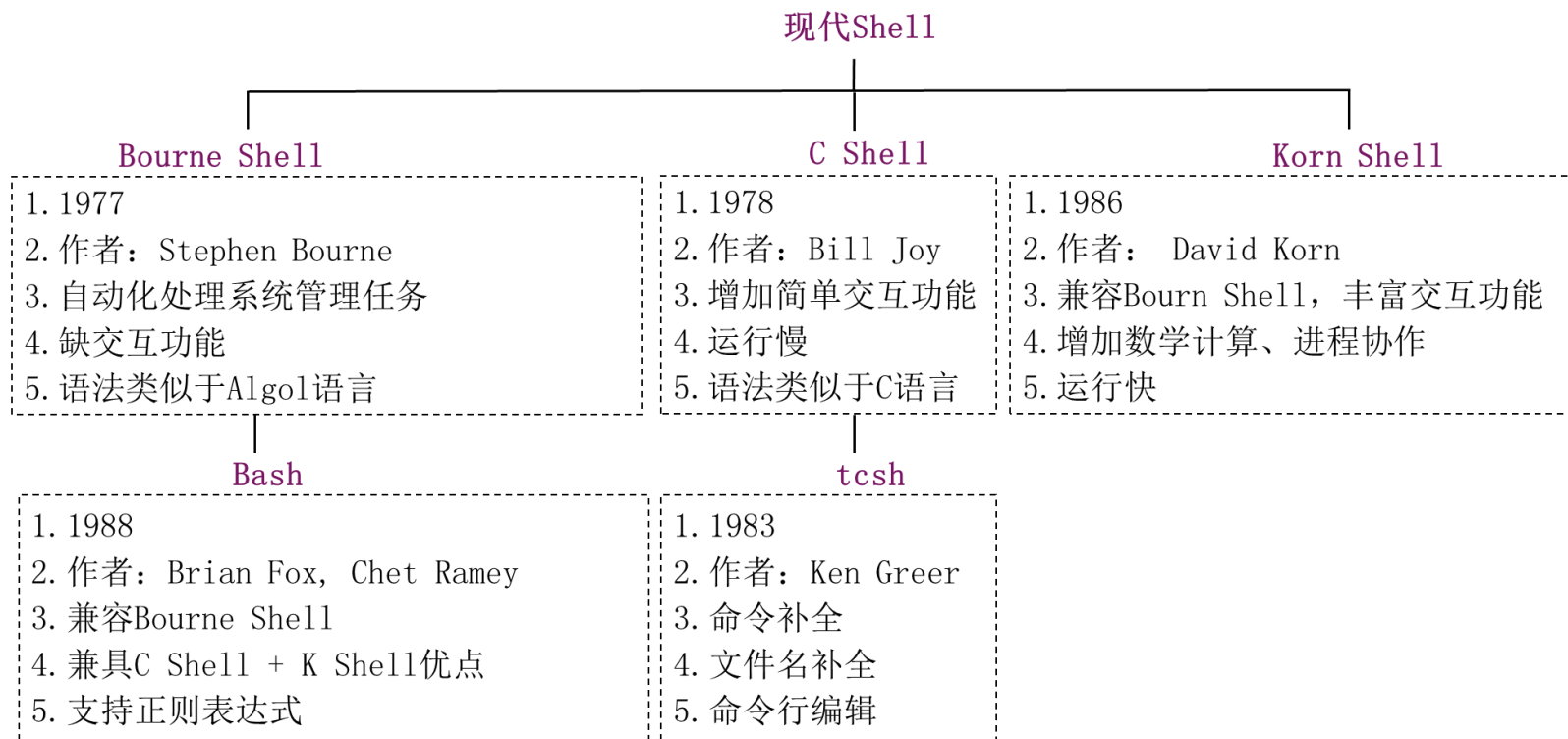
开发环境与工具 - SHELL

1.3.1 SHELL

**Linux缺省配置为
Bash**

**openEuler遵循
Linux通用标准**

现代常用Shell的特征概览



本课程实验环境: 使用Bash解释SHELL命令



1.3

SHELL - 命令



命令	作用
#useradd yuhong #passwd yuhong	创建新用户
\$mkdir sysprog \$cd sysprog	创建并进入目录sysprog
\$touch greeting.sh \$chmod 755 greeting.sh	创建并设置文件权限
\$ls -al	显示目录完整列表（含隐藏文件）
\$wc -lwc /etc/passwd	统计 /etc/passwd 文件的行数、字符数和单词数
\$uname -m	查看计算机硬件架构
\$last -n 6 root	显示 root 用户最近6条记录
\$who awk '{print \$1}' uniq 或 \$who tr -s ' ' cut -d ' ' -f 1 sort -u	统计当前所有不重复的在线用户名
\$sudo dmesg more	分页显示开机信息
\$tail -10 /etc/passwd 或 \$tail -n 10 /etc/passwd	显示/etc/passwd/文件的最后10行内容



1.3

SHELL - 命令



命令	作用	
\$tail -n +10 /etc/passwd cat -n	显示 /etc/passwd 文件第10行至末尾的内容（带行号）	
\$rpm -qa grep gcc	检查系统是否安装了gcc（GNU编译器套件）	
\$sudo rpm -ivh man-pages-3.53-5.el7.noarch.rpm	1) 下载对应版本的 RPM 包(如 man-pages-3.53-5.el7.noarch.rpm ^[1])；2) 执行安装命令	安装 man 手册页
\$sudo yum -y install man-pages	使用 yum 包管理器安装	
\$tar czvf yuhong_packs.tar.gz ./.*	压缩打包当前目录所有文件(排除子目录)	创建并提取当前目录的归档压缩包
\$tar xzvf yuhong_packs.tar.gz	解压并提取所有文件	
\$find . -type f -perm 644 -exec ls -l {} \;	查找当前目录中权限为 644（即属主可读可写，属组和其他用户只可读）的文件	
\$find ./ -name "*.c" -exec grep "gets(" {} \; -print	在当前目录及其子目录中查找	查找当前目录中调用了 gets() 函数的 C 语言源文件
\$grep -l "gets(" *.c	只查当前目录	
\$find ./ -name "admin.log[0-9][0-9][0-9]" -atime -7 -ok rm {} \;	删除当前目录中最近7天内访问过的、以 admin.log 为前缀且带数字后缀的日志文件	



1.3.1

SHELL - 脚本基础



```
1 #!/bin/bash
2 # sum1.sh: 用while循环计算1+2+...+100并打印结果

3 index=1
4 sum=0

5 while [ $index -le 100 ];
6 do
7   sum=`expr $sum + $index`
8   index=`expr $index + 1`
9 done

10 echo "The result of (1+2+3+...+100) is $sum"
```

```
1 #!/bin/bash
2 # sum2.sh: 用for循环计算1+2+...+100并打印结果

3 sum=0

4 for (( index=1; index<=100; index++ ))
5 #或: for index in {1..100}
6 #或: for index in `seq 100`
7 do
8     sum=`expr $sum + $index`
9 done
```

源代码 (sum1.sh) : 用while循环实现

源代码 (sum2.sh) : 用for循环实现

Shell脚本的执行

```
$ chmod u+x sum1.sh #为脚本添加执行权限
$ ./sum1.sh
```

```
$ bash ./sum1.sh #可不改变脚本文件的权限
$ bash -x ./sum1.sh #使用调试模式运行脚本
```



1. 用ls -al 命令列出下面的文件列表，是符号连接文件的是（ ）。

- A. -rw-rw-rw- 2 hel-s users 56 Sep 09 11:05 hello
- B. -rwxrwxrwx 2 hel-s users 56 Sep 09 11:05 goodbye
- C. drwxr--r-- 1 hel users 1024 Sep 10 08:10 zhang
- D. lrwxr--r-- 1 hel users 7 Sep 12 08:12 cheng

2. 文件exer1的访问权限为rw-r--r--，现要增加所有用户的执行权限和同组用户的写权限，下列命令中能满足的是（ ）。

- A. chmod a+x, g+w exer1
- B. chmod 765 exer1
- C. chmod o+x exer1
- D. chmod g+w exer1



1.3.2

C语言开发环境



检查关键工具

```
$which gcc gdb make  
#或  
$rpm -q gcc gdb make
```

安装开发环境

```
$ sudo yum groupinstall "Development Tools" #安装基础开发工具组  
$ sudo yum install kernel-devel #安装内核开发库  
$ sudo yum install glibc-devel #安装C/C++开发库
```

1.3.2.1 编译程序

```
$ gcc [-c/-S/-E] [-std=standard]  
      [-g] [-pg] [-Olevel]  
      [-Wwarn...] [-Wpedantic]  
      [-Idir...] [-Ldir...]  
      [-Dmacro[=defn]...] [-Umacro]  
      [-foption...] [-mmachine-option...]  
      [-o outfile] [@file] infile...
```




1.3.2.1

编译程序 - 不要忽视警告信息



请问以下代码有问题吗？

```
1 #include <stdio.h>
2 int main()
3 {
4     char array[256];
5     char index[2] = {-1, 'Z'};
6     for (int i = 0; i < 256; i++) {
7         array[i] = i;
8     }
9     printf("a[index[0]] = %d\n", array[index[0]]);
10    return 0;
11 }
```

源代码: charIndex.c

```
$ gcc -o charIndex charIndex.c
```



1.3.2.1

编译程序 - 不要忽视警告信息



```
1 #include <stdio.h>
2 int main()
3 {
4     char array[256];
5     char index[2] = {-1, 'Z'};
6     for (int i = 0; i < 256; i++) {
7         array[i] = i;
8     }
9     printf("a[index[0]] = %d\n", array[index[0]]);
10    return 0;
11 }
```

```
$ gcc -Wall -o charIndex charIndex.c
charIndex.c: 在函数 ‘main’ 中:
charIndex.c:9:35: 警告: 数组下标类型为 ‘char’ [-Wchar-subscripts]
printf("a[index[0]] = %c\n", array[index[0]]);
```

良好的编译习惯: `$gcc -Wall ...`

更好的编译习惯: `$gcc -Wall -ansi/pedantic...#跨平台`



1.3.2.2

调试程序：（1）查看全局变量errno



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <errno.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <sys/fcntl.h>
7  int main(int argc, char *argv[])
8  {
9      char *fName="test.txt";
10     int fd = open(fName, O_RDWR);
11     if ((fd = open(fName, O_RDWR, 0644)) == -1) {
12
13         perror("Fail to open file: ");
14         printf("Fail to open %s because of %s\n", fName, strerror(errno));
15         return(-1);
16     }
17     close(fd);
18 }
```

源代码: useErrno.c

```
$ gcc -Wall -o useErrno useErrno.c
```

```
$ ./useErrno
```

获得出错的“因”

```
Fail to open file: : Permission denied
```

```
Fail to open test.txt because of Permission denied
```

```
$ ls -l test.txt
```

```
-r----- 1 yuhong yuhong 25  1月 31 12:18 test.txt
```

```
perror("Fail to open file: ");
```

```
printf("Fail to open %s because of %s\n", fName, strerror(errno));
```

```
return(-1);
```

But, must think about applications. Not always appropriate to exit when something goes wrong.

-- 《computer systems a programmer perspective》



1.3.2.2

调试程序 - POSIX错误码规范(errno)



● 位置

/usr/include/asm-generic(errno.h

● 命名规范: 以E开头

● 取值范围

非0正整数值

● 全局性

每个线程/进程有独立的 errno 副本

● 时效性: 仅当系统调用失败时才有效

成功时值未定义 (保留的是前一错误调用的值)

必须立即检查, 避免被后续调用覆盖

● 错误分类与示例

按系统调用功能分组

open()调用的错误码:25种

联机帮助节选: 系统调用open()

...

RETURN VALUE

open() and creat() return the new file descriptor, or -1 if an error occurred (in which case, errno is set appropriately).

ERRORS

EACCES The requested access to the file is not allowed, or search permission is denied for one of the directories in the path prefix of pathname, or the file did not exist yet and write access to the parent directory is not allowed. (See also path_resolution(7).)

EDQUOT Where O_CREAT is specified, the file does not exist, and the user's quota of disk blocks or inodes on the file system has been exhausted.

EEXIST Pathname already exists and O_CREAT and O_EXCL were used.

...

```
$man 2 open | sed 's/[ ] [ ]*//g' | cut -d ' ' -f1 | grep ^E | wc -l
```



1.3.2.2

调试程序：（2）查看系统调用或库函数的 man手册页的描述



```
1 #include <stdlib.h>
2 #include <string.h>
3 int main(int argc, char *argv[])
4 {
5     char *dest="echo ";
6     system(strcat(dest,"Hello world!"));
7 }
```

源代码：useMan.c

```
$ gcc -Wall -o useMan useMan.c
$ ./useMan
段错误（核心已转储）
```

```
1 #include <stdlib.h>
2 #include <string.h>
3 int main(int argc, char *argv[])
4 {
5     char *dest=malloc(5+12+1);
6     dest = strcpy(dest,"echo ");
7     system(strcat(dest,"Hello world!"));
8 }
```

源代码：
CorrecteduseMan.c

STRCAT(3) Linux Programmer's Manual STRCAT(3)

NAME

strcat, strncat - concatenate two strings

SYNOPSIS

```
#include <string.h>
char *strcat(char *dest, const char *src);
char *strncat(char *dest, const char *src, size_t n);
```

DESCRIPTION

The `strcat()` function appends the src string to the dest string, overwriting the terminating null byte (`'\0'`) at the end of dest, and then adds a terminating null byte. The strings may not overlap, and the dest string must have enough space for the result. If **dest** is not large enough, program behavior is unpredictable; buffer overruns are a favorite avenue for attacking secure programs.

The `strncat()` function is similar, except that

- * it will use at most `n` bytes from src; and
- * src does not need to be null-terminated if it contains `n` or more bytes.

As with `strcat()`, the resulting string in dest is always null-terminated.

If src contains `n` or more bytes, `strncat()` writes `n+1` bytes to dest (`n` from src plus the terminating null byte). Therefore, the size of dest must be at least `strlen(dest)+n+1`.

A simple implementation of `strncat()` might be:

```
char* strncat(char *dest, const char *src, size_t n)
{
    size_t dest_len = strlen(dest);
    for (size_t i = 0 ; i < n && src[i] != '\0' ; i++)
        dest[dest_len + i] = src[i];
    dest[dest_len + i] = '\0';
    return dest;
}
```

...



1.3.2.2

调试程序：（3）GDB



```
$ gcc -Wall -g -o useMan useMan.c
```

```
1 $ gdb ./useMan
...
Reading symbols from ./useMan...
2 (gdb) l 6,6
3      6      system(strcat(dest,"Hello world!"));
4 (gdb) b 6
5 Breakpoint 1 at 0x400690: file ./useMan.c, line 6.
6 (gdb) r
7 ...
8 Breakpoint 1, main (argc=1, argv=0xffffffff9e8) at ./useMan.c:6
9      6      system(strcat(dest,"Hello world!"));
10 (gdb) x/10i $pc
11 => 0x400690 <main+28>: ldr     x0, [x29, #40]
12      0x400694 <main+32>: bl     0x400530 <strlen@plt>
13      0x400698 <main+36>: mov     x1, x0
14      0x40069c <main+40>: ldr     x0, [x29, #40]
15      0x4006a0 <main+44>: add     x2, x0, x1
16      0x4006a4 <main+48>: adrp    x0, 0x400000
17      0x4006a8 <main+52>: add     x1, x0, #0x780
18      0x4006ac <main+56>: mov     x0, x2
19      0x4006b0 <main+60>: ldr     x2, [x1]
20      0x4006b4 <main+64>: str     x2, [x0]
21 (gdb) ni 9
22 0x00000000004006b4      6      system(strcat(dest,"Hello world!"));
23 (gdb) x/3i $pc
24 => 0x4006b4 <main+64>: str     x2, [x0]
25      0x4006b8 <main+68>: ldur    x1, [x1, #5]
26      0x4006bc <main+72>: stur    x1, [x0, #5]
27 (gdb) p/a $x0
28 $1 = 0x40077d
29 (gdb) maint info section
30 Exec file:
    ~/home/szu/sysprog/Shell/Ccode/debugCode/useMan', file type elf64-
    littleaarch64.
31 ...
32 [14] 0x00400770->0x0040078d at 0x00000770: .rodata ALLOC LOAD READONLY DATA
    HAS_CONTENTS
33 ...
34 (gdb)
```

	命令	功能
加载	file fName	加载可执行目标文件fName
	r[un] [args]	运行程序，参数为args（若其非空）
	attach pid	连接并调试指定ID的进程。
终止	kill	终止正在调试的程序
	quit	终止gdb
help topic	帮助主题，如，可通过help breakpoints/break/brea/bre/br/b命令查看关于断点这一帮助主题的所有信息	
断点管理	b[reak] [fName] location	在指定位置设置断点，程序运行到该点时将暂停，location可以是16进制地址、函数名、行号、相对行位移等。
	[r]watch expr	为表达式/变量expr设置观察点，程序读/写expr时，执行暂停
	info break[points]	列出当前所有的断点
	clear [location]	去除指定的断点或所有断点
跟踪执行	c[ontinue]	从断点开始继续执行
	s[tep]	执行下一行代码，如遇函数调用，则进入该函数
	n[ext]	执行下一行代码，如遇函数调用，将其作为一步直接执行完毕
	ni	执行下一条机器指令
	finish	自动执行完当前函数，返回时打印堆栈地址、返回值及参数值等
查询	l[list] [location]	查看location附近（默认10行）的源代码
	search regexpr	根据正规表达式regexpr搜索源代码
	b[ack]t[race] [-full] [n]	n为正数：最外层的n个函数调用栈帧；n为负数：打印最内层 n 个栈帧。添加-full选项，可同时打印局部变量的值
	where	定位进程终止前正在运行的代码行，常用于分析崩溃（如核心转储文件）时确定故障点的重要操作
	info [what]	显示程序当前的调试信息，包括但不限于断点、局部变量、函数参数
	p[rint] [expr]	打印变量或表达式expr的当前值
	disass[emble] funcName	获取函数funcName的汇编代码
	maint info section	显示gdb内部维护的所有节信息，如.text、.data等各节的加载地址、文件偏移及属性
	info proc mappings	获取运行期每节的实际内存映射
	x/<n f u> addr	查看内存地址中的值
修改	set name expr	给变量或参数赋值
	j[ump] location	让程序立即流跳转到指定位置(如行号或地址)继续执行



1.3.2.2

调试程序 –基于核心转储 (core dump) 文件的GDB调试



1.使用包含-g选项的命令编译源程序

```
$gcc -Wall -g -o useMan useMan.c
```

2.将核心转储文件的大小限制设置为无上限

```
$sudo ulimit -c unlimited
```

3.设置核心转储文件的命名规范和存储位置

```
$sudo sysctl -w kernel.core_pattern=`pwd`/core-%e.%p.%h.%t
```

4.运行异常终止的可执行文件，生成核心转储文件

```
$. /useMan
```

5.用gdb命令调试核心转储文件

```
$gdb -c core-useMan.6250.taishan02-vm-10.1612438919
```

或

```
$gdb ./useMan core-useMan.6250.taishan02-vm-10.1612438919
```

或

```
$gdb ./useMan
```

```
...
```

```
(gdb) core-file core-useMan.6250.taishan02-vm-10.1612438919
```

6.调试完后，恢复系统缺省设置：不生成核心转储文件

```
$sudo ulimit -c 0
```

```
1 GNU gdb (GDB) EulerOS 8.3.1-11.oe1
...
2 For help, type "help".
3 Type "apropos word" to search for commands related to "word".
4 [New LWP 412882]
5 Missing separate debuginfo for the main executable file
6 Try: dnf --enablerepo='*debug*' install /usr/lib/debug/.build-
id/4b/b4750ac5965d7f8f76da4473311007e08cbb4d
7 Core was generated by './useMan'.
8 Program terminated with signal SIGSEGV, Segmentation fault.
9 #0 0x0000000004006b4 in ?? ()

10 (gdb) symbol-file ./useMan
11 Reading symbols from ./prodCorefile...
12 (gdb) bt
13 #0 0x0000000004006b4 in main (argc=1, argv=0xffffed567d78)
14   at useMan.c:6
15 (gdb) where
16 #0 0x0000000004006b4 in main (argc=1, argv=0xffffed567d78)
17   at useMan.c:6
18 (gdb)
```

也可用file ./useMan



1.3.2.2

调试程序 – (4) 其他



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 void greeting(int nSize)
6 {
7     char *p = (char *)malloc(nSize * sizeof(char));
8     if (nSize < 12 || p == NULL) return;
9     strcpy(p, "Hello world");
10    printf("%s\n", p);
11    free(p);
12 }
13 void run()
14 {
15     sleep(0);
16     return;
17 }
18 void service() {
19     greeting(11);
20     run();
21     return;
22 }
23 int main(int argc, char *argv[])
24 {
25     while(1) {
26         service();
27     }
28     return 0;
29 }
```

源代码 (greeting-service.c)

```
$ ps aux | grep PID; ps aux | grep greeting
USER PID %CPU    %MEM    VSZ   RSS_TTY STAT  START       TIME COMMAND
...
yuhong 104916 3.0    0.0    8512   5056 pts/0 S+ 14:08      0:00 ./greeting
yuhong 104916 2.1    12.3  863296 858304 pts/0 S+ 14:08      0:31 ./greeting
yuhong 104916 2.1    18.1 1271488 1268032 pts/0 S+ 14:08      0:45 ./greeting
yuhong 104916 2.1    25.5 1788736 1783360 pts/0 S+ 14:08      1:03 ./greeting
yuhong 104916 2.0    32.9 2303104 2298688 pts/0 S+ 14:08      1:22 ./greeting
```

● 内存泄漏

1.静态分析工具如mtrace

2.通过编译器使用地址消毒技术如ASAN、TSAN、TECH-SAN[1]等

#Asan

```
$gcc -fsanitize=address -g -o gs greeting-service.c
```

#Tsan

```
$gcc -fsanitize=thread -g -o threads threads.c -lpthread
```

3.动态分析工具如valgrind

[1] Y. Cao, Y. Feng, H. Li, C. Huang, F. Jian, H. Li, and X. Wang. 2025. Tech-ASan: Two-stage check for Address Sanitizer. In Proceedings of the 16th International Conference on Internetworkare (Internetworkare '25), pp. 96 - 107.



1.3.2.3

构建函数库 – 静态库



```
1 #define OSCHATBOX_DEGUB    0
2 /* 调试信息，编译时开启该选项则输出 */
3 ...
3 int Log(int loglevel, const char *msg,
        const char *fileName);
```

源文件(logger.h): OSChatBox日志事件级别定义

```
1 #include "logger.h"
2 ...
3 int Log(int loglevel, const char *msg,
        const char *fileName) { ... }
```

源文件(logger.c): OSChatBox写日志事件

```
1 $ ls
2 logger.c  logger.h  oschatbox.log
3 $ gcc -Wall -c logger.c
4 $ ls
5 logger.c  logger.h  logger.o  oschatbox.log
6 $ ar rcs liblog.a logger.o
7 $ ls
8 liblog.a  logger.c  logger.h  logger.o
   oschatbox.log
```

```
1 #include <stdio.h>
2 #include "logger.h"
3 int main(int argc, char *argv[])
4 {
5     Log(0, "A test.", "oschatbox.log");
6 }
```

源文件 (testlog.c) : 使用静态库函数Log()

```
1 $ gcc -Wall -static -o testlog_static testlog.c -L. -llog
2 $ ./ testlog_static
```

源文件 (testlog.c) : 编译 & 运行

使用静态库编码

■ 优点

可执行目标代码运行环境
依赖低，跨平台兼容性好

■ 缺点

- 1.内存与加载效率低
- 2.磁盘存储冗余
- 3.更新维护繁琐



1.3.2.3

构建函数库-动态库-动态链接



```
...  
int Log(int loglevel, const char *msg, const char *fileName);
```

源文件(logger.h): OSChatBox日志事件级别定义

```
1 #include "logger.h"  
2 ...  
3 int Log(int loglevel, const char *msg, const char *fileName) {...}
```

源文件(logger.c): OSChatBox写日志事件

```
1 $ gcc -Wall -fPIC -c logger.c  
2 $ ls  
3 logger.c logger.h logger.o oschatbox.log  
4 $ gcc -shared -o liblog.so logger.o  
5 $ ls  
6 liblog.so logger.c logger.h logger.o oschatbox.log
```

构建静态库文件

```
...  
1 int main(int argc, char *argv[])  
2 {  
3     Log(0, "A test.", "oschatbox.log");  
4 }
```

源文件 (testlog.c) : 使用动态库函数Log()

```
1 $ gcc -o testlog_dynamic -llog -L.  
testlog.c  
2 $ ls  
3 testlog.c liblog.so logger.h  
oschatbox.log testlog_dynamic logger.c  
logger.o  
4 $ echo $LD_LIBRARY_PATH  
5 $ ./testlog_dynamic  
6 ./ testlog_dynamic: error while loading  
shared libraries: liblog.so: cannot open  
shared object file: No such file or  
directory  
7 $ export LD_LIBRARY_PATH=`pwd`  
8 $ ./ testlog_dynamic  
9 $ cat oschatbox.log  
10 OSChatBox debug: A test.
```

使用动态库编码

- 立即绑定: 环境变量LD_BIND_NOW=0
- 延迟绑定: 环境变量LD_BIND_NOW=1

优点

- 1.内存、磁盘存储效率高
- 2.减少加载时间
- 2.更新维护方便

缺点

链接或运行时均需共享库文件存在



1.3.2.3

构建函数库-动态库-动态加载



```
1 #include <stdio.h>
2 #include <dlfcn.h>
3 #include <stdlib.h>
4 int dolog(char *lib, char *method, int arg0, char *arg1, char *arg2)
5 {
6     void *dl_handler;
7     int (*func)(int arg0, char *arg1, char *arg2);
8     char *symr_error;
9     dl_handler = dlopen(lib, RTLD_LAZY); //以延迟绑定模式打开共享库
10    if (!dl_handler) return (-1);
11    func = dlsym(dl_handler, method); //符号解析
12    symr_error = dlerror(); //返回符号解析错误
13    if (symr_error != NULL) return (-1);
14    return ((*func)(arg0, arg1, arg2);); //调用Log()
15 }
16 int main(int argc, char *argv[])
17 {
18     return (dolog("./liblog.so", "Log", 0, "dl test.", "oschatbox.log "));
19 }
```

函数库分类

	静态链接时	加载时	运行时	时间
函数库	静态库	✓ 符号解析		
		✓ 重定位		
	共享库	✓ 符号解析	✓ 重定位 (LD_BIND_NOW = 1)	✓ 重定位 (LD_BIND_NOW = 1)
		✓ 隐式使用库		
	动态加载	✓ 显式使用库	✓ 符号解析 ✓ 重定位 (RTLD_NOW)	✓ 符号解析 ✓ 重定位 (RTLD_LAZY)

```
$ gcc [-rdynamic] -o dl_app dl_app.c -ldl
```

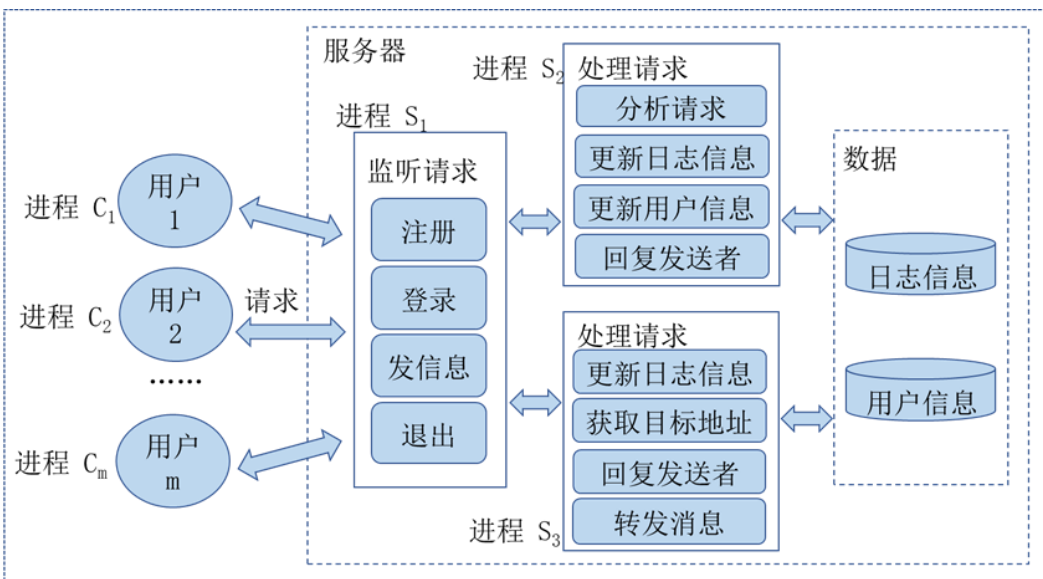


1.4

现场即时通信系统软件OSChatBox初始化



参考系统模型

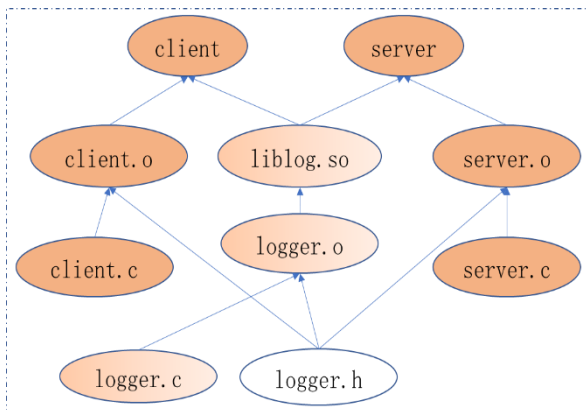


```
$ ls -R
.:
bin headers lib log Makefile obj src
./bin:
client server
./headers:
logger.h
./lib:
liblog.so
./log:
client.log server.log
./obj:
client.o logger.o server.o
./src:
client libsrc server
./src/client:
client.c
./src/libsrc:
Log Makefile
./src/libsrc/Log:
testlog testlog.c logger.c testlog.log
./src/server:
server.c
```



1.4

现场即时通信系统软件OSChatBox初始化



文件依赖关系DAG图

```
1 .PHONY: clean #避免当前路径下存在文件clean致其不能被执行
2 CC = gcc
3 CFLAGS = -O -Wall
4 CFLAGS-L = -O -Wall -fPIC
5 INCLUDE = -I ./headers/
6 LIBADDR = -L ./lib/ #指明liblog.so的路径
7 SEROBJ = ./obj/server.o
8 CLI OBJ = ./obj/client.o
9 LOGOBJ = ./obj/logger.o
10 LOGOBJ-S = ./obj/s-logger.o
11 LOG-SO = ./lib/liblog.so
12 LOG-A = ./lib/liblog.a
```

参考工程文件源代码 (Makefile)

```
13 all: server client $(LOG-SO) $(LOGOBJ) $(LOG-A) $(LOGOBJ-S)
14 server: $(SEROBJ) $(LOG-SO) $(LOGOBJ)
15         $(CC) $(CFLAGS) ^ -o ./bin/$@ -llog $(LIBADDR)
16 client: $(CLI OBJ) $(LOG-SO) $(LOGOBJ)
17         $(CC) $(CFLAGS) ^ -o ./bin/$@ -llog $(LIBADDR)
18 $(LOG-SO) $(LOGOBJ): $(LOGOBJ)
19         $(CC) -shared -o $@ $%
20 $(LOG-A) $(LOGOBJ-S): $(LOGOBJ-S)
21         ar cr $@ $%
22 $(SEROBJ): ./src/server/server.c
23         $(CC) $(CFLAGS) -c $^ -o $@ $(INCLUDE) -llog $(LIBADDR)
24 $(CLI OBJ): ./src/client/client.c
25         $(CC) $(CFLAGS) -c $^ -o $@ $(INCLUDE) -llog $(LIBADDR)
26 $(LOGOBJ): ./src/libsrc/Log/logger.c
27         $(CC) $(CFLAGS-L) -c $^ -o $@ $(INCLUDE)
28 $(LOGOBJ-S): ./src/libsrc/Log/logger.c
29         $(CC) $(CFLAGS) -c $^ -o $@ $(INCLUDE)
30 clean-lib:
31         rm -rf $(LOGOBJ) $(LOGOBJ-S) ./lib/*.o
32 clean:
33         rm -rf $(SEROBJ) $(CLI OBJ) ./bin/*
34         >./log/server.log; >./log/client.log
```

`$ make [all / server / client / clean / clean-lib]`



课程内容简介

围绕现场即时通信系统软件OSChatBox的构建展开，项目式教学

理论知识

进程控制： 系统调用，僵尸进程/孤儿进程/守护进程.....

进程间通信： 管道、命名管道、信号.....

多线程编程： 多核处理器架构、POSIX线程控制函数、线程属性、线程安全

5个基本实验

C开发环境与工具（选作）

进程创建与终止

进程同步与守护进程

进程间通信

多线程编程

期末大作业（任选其一完成）

选题一“功能迭代”： 拓展OSChatBox的功能或优化其性能

选题二“开源贡献”： 阅读内核代码，发现、修复问题并向上游社区提交补丁

选题三“应用创新”： 基于教师科研项目，应用课程知识优化

成绩组成

实验：

4次 ($10\% * 4 = 40\%$)

闭卷考试：

1次 (10%)

大作业 – 50%

课堂表现（加分项）



本章小结与实验



关键知识点回顾

- ❖ 系统调用:用户程序与内核的桥梁
- ❖ 系统编程:接近底层的编程范式
- ❖ 开发工具:GCC、GDB、Make的熟练使用
- ❖ 项目实践:OSChatBox工程初始化

课后任务

- ❖ 必做: 完成openEuler实验环境搭建, 成功编译、运行、调试一个程序
- ❖ 选做: 构建OSChatBox工程文件
- ❖ 拓展: 阅读《系统编程原理与实践》第一章并选作课后练习

下章预告

- ❖ 进程控制系统调用: 学习创建、终止、同步进程的编程接口与内核原理。
- ❖ 特殊进程剖析: 分析孤儿、僵尸和守护进程的形成与特征, 学会避免僵尸进程和构建守护进程的通用流程
- ❖ 综合实践: 将 OSChatBox 服务器实现为一个健壮的守护进程。
- ❖ 进程的资源统一视图: 从用户态的文件描述符, 到内核态文件对象与inode 结构

参考资料

教材

- ❖ 冯禹洪, 系统编程原理与实践, 待出版

参考教材

- ❖ W.Richard Stevens, Stephen A.Rago, UNIX环境高级编程 (第3版), 尤晋元 张亚英 戚正伟 译, 人民邮电出版社
- ❖ Mark L. Mitchell, Alex Samuel, Jeffrey Oldham, Advanced Linux Programming, New Riders
- ❖ Robert Love, Linux System Programming, O'Reilly Taiwan公司译, 东南大学出版社出版
- ❖ C语言编程实战
- ❖ openEuler + glibc 相关源代码